AstraQuantum Tech

AQT · SUMMER OF CYBERSECURITY 2026

WEEK 09 · PRACTICAL ASSIGNMENT REPORT

Malware Analysis & Static Triage

VirusTotal | Strings | PEStudio | PE Headers | Threat Intelligence

SUBMITTED BY
Muhammad Umer Imran

STUDENT ID
AQT-1170

INSTRUCTOR
Husnain Shahid

DATE
16 October 2026

1 Executive Summary

VERDICT	SAMPLE	VIRUSTOTAL	LEARNING
Malicious	**Chrome extension**	**Not found**	**2 rooms + 1 lab**
High confidence, from code	Manifest V2, "ChatGPT"	Hash never submitted	All completed

This report covers my Week 9 work: two TryHackMe rooms (MAL: Malware Introductory and MAL: Researching), the CyberDefenders **FakeGPT** blue-team lab, and a static triage of the FakeGPT sample using hashes, VirusTotal, and plain-text reading of its files. All of it was done without running the sample.

The FakeGPT sample is a Chrome extension (Manifest V2) that calls itself "ChatGPT" but behaves like an information stealer. Its code reads the username and password from form submissions, records every keystroke, encrypts the data with AES using a key hard-coded in the script, and sends it to `mo[.]elshaheedy[.]com` by adding a hidden image whose URL carries the stolen data. The manifest asks for far more access than a chat helper needs (all sites, cookies, tabs, blocking web requests). My overall assessment is **malicious, high confidence**, and that confidence comes from the code itself. VirusTotal had *no record* of the package hash, so reputation data adds nothing here, and I treat that as neutral rather than as a sign the file is safe.

2 Scope & Safety Statement

I only worked with material supplied by the course platforms: the TryHackMe lab VMs and the FakeGPT lab download from CyberDefenders (password-protected zip, as the lab page instructs). I did not run or install the extension, and I did not load any of its files in a browser. The files were only read as text. The hash of the zip was searched on VirusTotal by hash; nothing was uploaded, and I was not signed in. No dynamic analysis was done. No personal, university or client files were used, and no samples are shared with this report, only hashes and text indicators.

3 Sample Information

Item	Detail
Main sample	213-FakeGPT.zip (CyberDefenders "FakeGPT" lab, category Malware Analysis, difficulty Easy)
Size	144,536 bytes (Fig. 13); downloaded 16/10/2026, 11:55 AM
SHA256	`84A4F75606314D3FD487CBA1B8F60353FC4F5A4851DC02B9DEEB2EC34250472D`
Contents (6 items)	manifest.json, loader.js, app.js, crypto.js, ui.html, img.gif (Fig. 10)
What it is	Chrome browser extension, manifest_version 2, name "ChatGPT", version 1.0, description "An AI-powered assistant extension."
Practice files (THM VM)	aws.exe MD5 `D2778164EF643BA8F44CC202EC7EF157` LoginForm.exe MD5 `FF395A6D528DC5724BCDE9C844A0EE89` TryHackMe.exe SHA256 `6F870C80361062E8631282D31A16872835F7962222457730BC55676A61AD1EE0`

4 Tools Used

Tool	Used for
PowerShell `Get-FileHash`	SHA256 of the FakeGPT zip and of TryHackMe.exe
VirusTotal (web, hash search)	Reputation lookup of the zip hash
Notepad + Find	Reading manifest.json and the JavaScript; searching for strings such as "elshah"
HxD, HashTab, PEiD (THM VM)	Header bytes, MD5 values, packer check on practice files
CyberDefenders lab page	Scenario, questions and answers for the challenge (10/10 solved)

PEStudio is the course tool for PE inspection. A PEStudio shortcut is present in the THM "PE Tools" folder (Fig. 3), but the FakeGPT package contains no Windows executable for it to open, so the PE observations below come from HxD and PEiD on the lab files.

5 TryHackMe Room Summaries

MAL: Malware Introductory — completed (14 tasks, 176 points, Fig. 4)

This room starts with why malware analysis matters at all, then looks at how an analyst can tell that an attack has already happened, through host signs such as odd files and processes and network signs such as strange outbound connections. The main idea I took from it is the split between static analysis, where you study a file without running it, and dynamic analysis, where you run it and watch. The room is blunt that dynamic work is risky, because a ransomware sample will really encrypt your files. In the practical parts I opened a file in HxD to read its MD5 and its first bytes, and I used PEiD to check an executable for a packer. The last theory task explains obfuscation and packing, which authors use to stop people reading their code.

Three concepts: (1) static vs dynamic analysis; (2) host and network signatures show that an infection has happened; (3) packing hides the real code, and a tool like PEiD can flag known packers. **Question I still have:** if PEiD reports no packer, how sure can I be, given it only knows signatures?

MAL: Researching — completed (7 tasks, 136 points, Fig. 8)

This room is about checksums and how an analyst uses them to research a sample. A hash function turns any file into a fixed-length value, and changing even one character in the file gives a completely different value. That makes a hash useful for checking that a file copied correctly and for naming a sample without sharing it. I calculated hashes in three ways on the Windows VM: the File Hashes tab from HashTab (MD5 of LoginForm.exe), Get-FileHash in PowerShell (SHA256 of TryHackMe.exe) and the certutil syntax. The later tasks use the hash to look a sample up on VirusTotal and in public reports, which is the same thing I did afterwards with the FakeGPT package.

Three concepts: (1) a hash is fixed length and changes completely when the file changes; (2) MD5 is quick but weak against deliberate collisions, so SHA256 is the better choice for evidence; (3) a hash lookup is threat intelligence without executing anything. **Question I still have:** what should an analyst conclude when a hash has never been seen, as happened with my FakeGPT zip?

Dissecting PE Headers (the recommended extension) is not part of this submission. The PE work in this report comes from the PE tasks inside MAL: Malware Introductory.

6 Challenge Summary — CyberDefenders FakeGPT

Link: https://cyberdefenders.org/blueteam-ctf-challenges/fakegpt/ | **Status:** 10/10 questions, lab completed (Figs. 9 and 12).

Objective. The lab gives you a Chrome extension that presents itself as ChatGPT and asks what it really does. The job is to read the manifest and the scripts, work out what data it collects, how it hides that data and where it sends it, and answer ten questions. Everything is done by reading the files, so the extension never needs to be installed. The lab page lists the tactics as Credential Access, Collection, Command and Control and Exfiltration.



Diagram 1. Data path of the FakeGPT extension, reconstructed from manifest.json and the script in Figs. 11 and 15.

Indicators I found:

1. The manifest requests `tabs, http://*/*, https://*/*, storage, webRequest, webRequestBlocking, cookies`, runs a content script on `<all_urls>` and keeps a persistent background script (`system/loader.js`) (Fig. 11).
2. The script pulls `username` and `password` from submitted form data and packs them with the site name; a `keydown` listener sends every key pressed as type "keystroke" (Fig. 15).

3. Stolen data is encrypted with AES (CryptoJS) using the hard-coded key `SuperSecretKey123` and a random 16-byte IV, then Base64 encoded (Fig. 15).
4. It is sent by creating an `Image()` whose source is `https://Mo.Elshaheedy.com/collect?data=…` and appending it to the page. The lab answer for the exfiltration domain was `mo.elshaheedy.com`, and the answer for what the extension uses to handle session data was *cookies* (Fig. 12).

Conclusion. The extension steals credentials and keystrokes, hides them with encryption, and sends them out through an ordinary-looking image request. It is suspicious because an "AI assistant" has no reason to read every password field, log every key, or ask for cookie access on all sites, and because the name copies a well-known product.

7 Hash / VirusTotal Results

Object	Algorithm	Value	Where
213-FakeGPT.zip	SHA256	`84A4F75606314D3FD487CBA1B8F60353FC4F5A4851DC02B9DEEB2EC34250472D`	PowerShell, Fig. 13
TryHackMe.exe	SHA256	`6F870C80361062E8631282D31A16872835F7962222457730BC55676A61AD1EE0`	THM VM, Fig. 7
LoginForm.exe	MD5	`FF395A6D528DC5724BCDE9C844A0EE89`	HashTab, Fig. 6
aws.exe	MD5	`D2778164EF643BA8F44CC202EC7EF157`	HxD, Fig. 2

Searching the zip's SHA256 on VirusTotal returned **"Item not found"** (Fig. 14, 12:15 PM). So there are no vendor detections, no community comments and no first-seen date to use. This is expected for a small training sample that nobody has submitted, and it does not mean the file is clean. I did not upload the file because the rules say not to put samples on public services. A sensible next step in a real case would be to hash each extracted script and search those too.

8 Strings Findings

The FakeGPT files are JavaScript and JSON, so they are readable text and the Windows *strings* tool is not needed. I read them in Notepad and used Find to jump to interesting values (for example searching "elshah", Fig. 15). The readable strings that matter:

String	Why it stands out
`exfiltrateCredentials, exfiltrateData`	The function names say what the code does
`formData.get('password')`	Reads the password field on form submit
`'keystroke'`	Keylogging label used in the payload
`CryptoJS.AES.encrypt, SuperSecretKey123`	Encrypts stolen data; the key is hard-coded
`https://Mo.Elshaheedy.com/collect?data=`	Destination and parameter used to carry the data
`new Image(); img.src = …`	Image-request beacon, a way to send data that looks like loading a picture

9 PEStudio Findings

The FakeGPT package has no PE file, so PEStudio has nothing to parse. For PE structure I worked with the practice executables on the THM VM. In HxD, `aws.exe` begins with `4D 5A` ("MZ"), and the `50 45 00 00` ("PE") signature sits at offset `0xE0` (Fig. 2). That is the normal layout: DOS header first, then the NT headers further in. HxD also gave the MD5 (`D2778164EF643BA8F44CC202EC7EF157`). I did not see anything unusual in these header bytes.

10 PE Header / Packing Observations

PEiD was run on `AD29AA1B.bin` (path shown in Fig. 3, Task 9 folder). It reported **Microsoft Visual C++ 6.0**, entry point `00001820` in the `.text` section, first bytes `55 8B EC 6A`, linker 6.0 and subsystem Win32 console. An entry point inside the

normal code section and a standard compiler prologue (`push ebp; mov ebp, esp`) fit an unpacked program, so I saw no sign of packing in this file. This is only a clue: PEiD matches signatures, so a custom packer would not be named. For FakeGPT, the code I read was plain and readable, not packed. The lab description does mention anti-analysis techniques, so I would not claim the whole extension is free of obfuscation.

11 Indicators of Compromise (IOCs)

Type	Value	Source	Why it matters
SHA256	`84A4F75606314D3FD487CBA1B8F60353FC4F5A4851DC02B9DEEB2EC34250472D`	Get-FileHash, Fig. 13	Identifies the exact package; VirusTotal has no record
Domain	`mo[.]elshaheedy[.]com`	JS code, lab answer	Where stolen data is sent
URL	`hxxps://mo[.]elshaheedy[.]com/collect?data=`	JS code, Fig. 15	Request pattern to search in proxy logs
String	`SuperSecretKey123`	JS code, Fig. 15	Hard-coded AES key; also useful for searching and for decrypting captured data
Function names	`exfiltrateCredentials, exfiltrateData`	JS code, Fig. 15	Easy to hunt for in extension folders
Extension	"ChatGPT" v1.0, MV2, "An AI-powered assistant extension."	manifest.json, Fig. 11	Impersonates a trusted brand
Permissions	`tabs, http(s)://*/*, cookies, webRequest, webRequestBlocking`	manifest.json	Access far beyond the stated purpose
Files	`loader.js, app.js, crypto.js, ui.html, img.gif`	Fig. 10	File set of the package

12 Risk / Confidence Assessment

ID	Observation	Evidence and why it is relevant	Confidence	Impact / defensive action
MAL-01	Over-privileged extension posing as ChatGPT	Manifest (Fig. 11): all-site access, cookies, webRequestBlocking, persistent background. Permissions alone are only a clue; together with the borrowed name they are a strong warning.	Medium	Can read and alter any page. Allow-list extensions; review permission sets.
MAL-02	Credential and keystroke capture	Code reads username/password on submit and sends each keydown (Fig. 15). Direct evidence of intent.	High	Account takeover. Reset passwords and revoke sessions for anyone who installed it.
MAL-03	Encrypted payload, hard-coded AES key	CryptoJS AES with `SuperSecretKey123` (Fig. 15). Hides content from network inspection, but the fixed key lets defenders decrypt captured traffic.	High	Hunt for the key string; decrypt any captured requests during investigation.
MAL-04	Exfiltration through image requests	Hidden `Image()` to mo.elshaheedy.com/collect (Figs. 12, 15). Blends in with normal web traffic.	High	Block the domain; search proxy and DNS logs for it.
MAL-05	No VirusTotal record	"Item not found" (Fig. 14). Not evidence either way.	Low	Do not rely on reputation; rely on behaviour.
MAL-06	No packer seen in practice file	PEiD result and header bytes (Figs. 2, 3). Signature-based only.	Low–Medium	Confirm with section entropy and imports in PEStudio when available.

Strong evidence: the credential/keystroke code, the AES routine, the exfiltration URL and the lab's own answers.
Only clues: the permission list on its own, the extension name, the missing VirusTotal result, and the PEiD output.

13 Defensive Recommendations

1. Block `mo[.]elshaheedy[.]com` at DNS and proxy, and search logs for requests to `/collect?data=`.
2. Enforce a Chrome extension allow-list by policy so users cannot sideload unknown extensions; flag any extension named after a popular AI tool that is not from the official store.
3. Hunt endpoints for extension folders containing `exfiltrateCredentials` or `SuperSecretKey123`.
4. If a user ran it: reset passwords, revoke active sessions and cookies, and re-enrol MFA, since the extension could read all three.
5. Treat hash and domain as leads, correlate with other telemetry before closing the case, and record them in the IOC list.
6. Remind staff that fake “AI assistant” extensions are a common lure and to check permissions before installing.

14 Conclusion

From safe, observable evidence I can say that the FakeGPT package is a credential and keystroke stealer in the form of a Chrome extension. The proof is in its code, not in any tool’s verdict. VirusTotal knew nothing about it, which shows why one reputation lookup cannot decide a case. For the PE part I only saw clean, unpacked practice files, and I would need a real Windows sample and PEStudio output to say more about packing. What I learned is to combine hashes, strings and structure, say how sure I am, and then tell the SOC what to do next.

15 Appendix — Screenshots

All screenshots are from my own TryHackMe, CyberDefenders and laptop sessions on 16/10/2026. Date and time appear in the Windows taskbar where visible (the THM VM runs on its own clock).

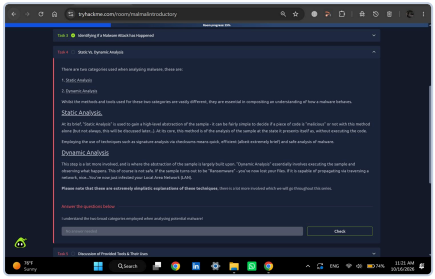


Fig. 1 MAL: Malware Introductory, Task 4 (Static vs Dynamic Analysis), room progress 25%. Taskbar: 11:21 AM, 10/16/2026.

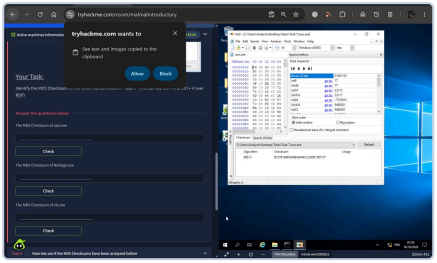


Fig. 2 HxD on aws.exe: MZ header at offset 0, PE signature at 0xE0, MD5 shown (THM VM clock 07:29, 16/10/2026).

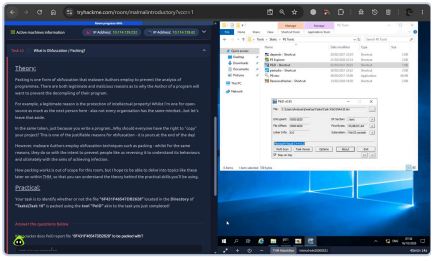


Fig. 3 PEiD on AD29AA1B.bin: Visual C++ 6.0, EP in .text; PE Tools folder with PEStudio shortcut (VM 07:36).

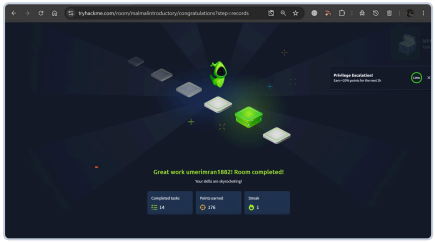


Fig. 4 MAL: Malware Introductory completed: 14 tasks, 176 points.

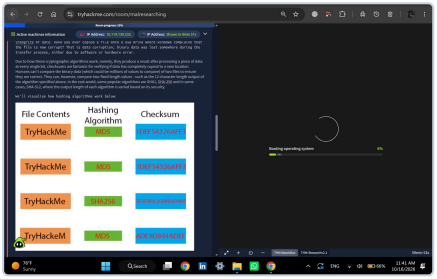


Fig. 5 MAL: Researching, Task 2: how hashing algorithms produce checksums. Taskbar: 11:41 AM.

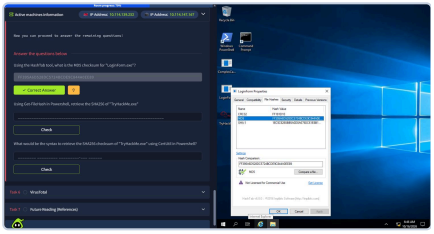


Fig. 6 HashTab File Hashes tab for LoginForm.exe (MD5, SHA-1, CRC32); answer accepted. VM 06:46.

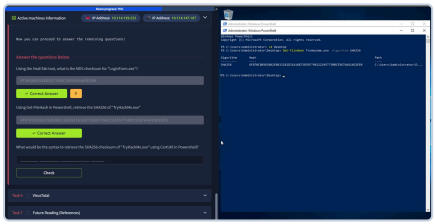


Fig. 7 Get-FileHash -Algorithm SHA256 on TryHackMe.exe in PowerShell; answer accepted.

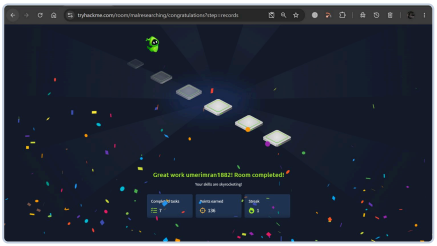


Fig. 8 MAL: Researching completed: 7 tasks, 136 points.

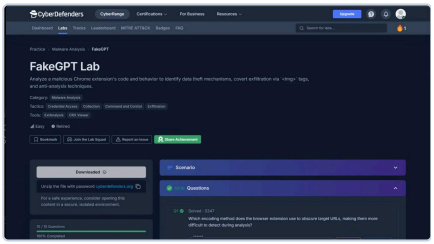


Fig. 9 CyberDefenders FakeGPT lab page: Easy, tactics and tools, 10/10 questions.

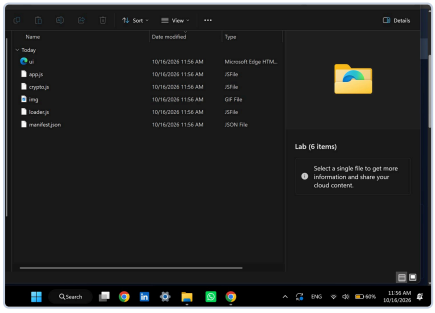


Fig. 10 Extracted FakeGPT package folder: 6 items (ui, app.js, crypto.js, img, loader.js, manifest.json). Taskbar: 11:56 AM.

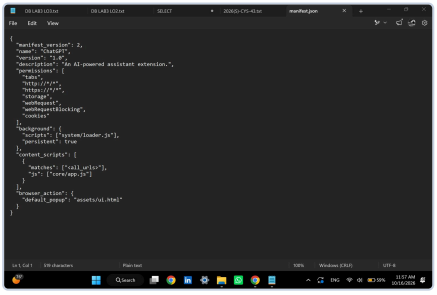


Fig. 11 manifest.json opened in Notepad: name, permissions, background and content scripts. Taskbar: 11:57 AM.

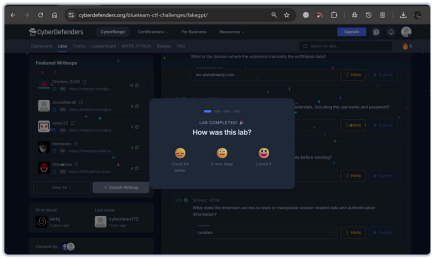


Fig. 12 FakeGPT lab completed; answers for the exfiltration domain and for ‘cookies’ visible.

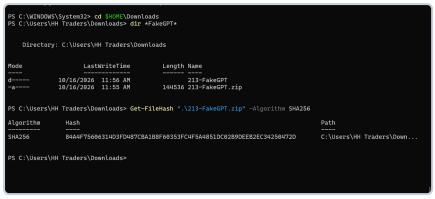


Fig. 13 Get-FileHash SHA256 of 213-FakeGPT.zip (144,536 bytes, 11:55 AM).

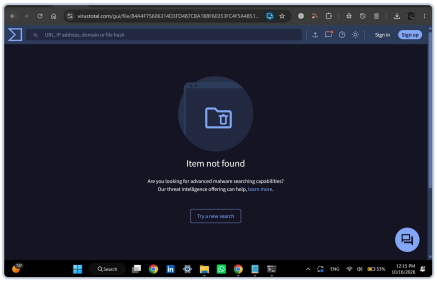


Fig. 14 VirusTotal search for the zip's SHA256: 'Item not found'. Taskbar: 12:15 PM.

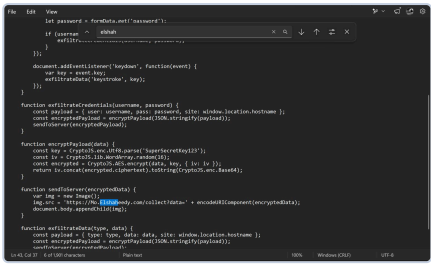


Fig. 15 JavaScript in Notepad: credential and keystroke capture, AES routine, image-beacon exfiltration; Find for 'elshah'.